Assignment-14.4

2303A51304

Batch-05

Task-01:

Prompt:

Create a fully working responsive startup homepage that runs immediately in Replit preview.

Create the main file named exactly "index.html".

Put all HTML and CSS inside this single file using a <style> section.

Do NOT create extra files.

The project must run automatically when the Run button is clicked in Replit.

Ensure no external dependencies are required.

WEBSITE REQUIREMENTS:

1. Header

Company logo/name

Navigation links: Home, About, Services, Contact ,Responsive navigation layout

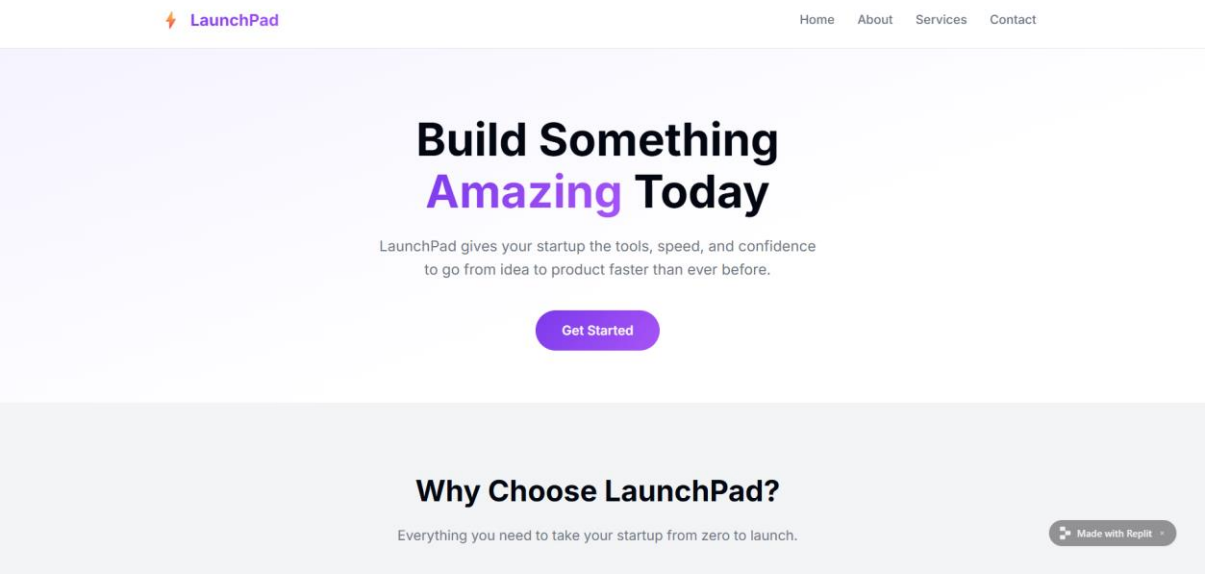
2. Main Section:Hero section with heading, description, and button

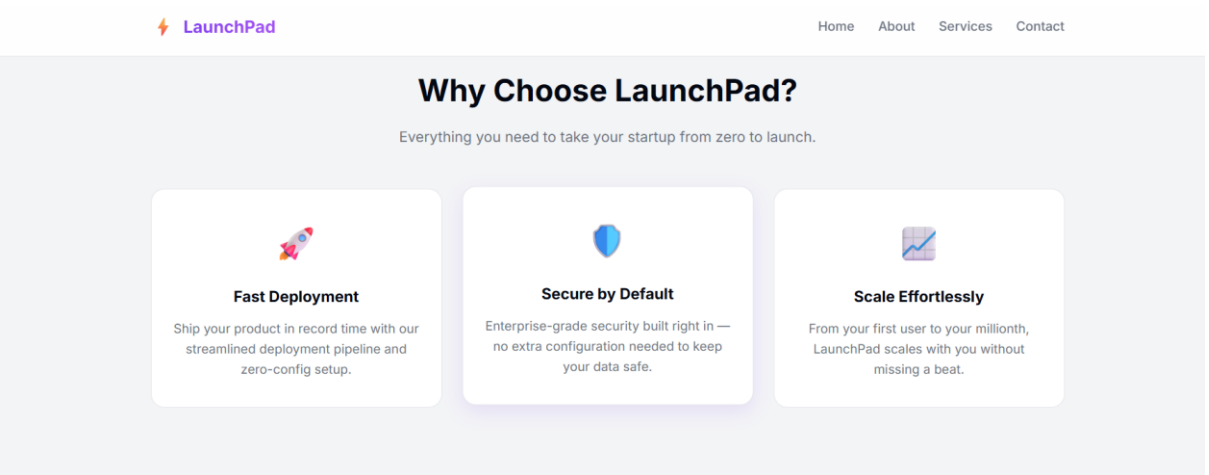
Three feature cards

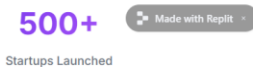
OUTPUT FORMAT: Return only ONE complete index.html file.

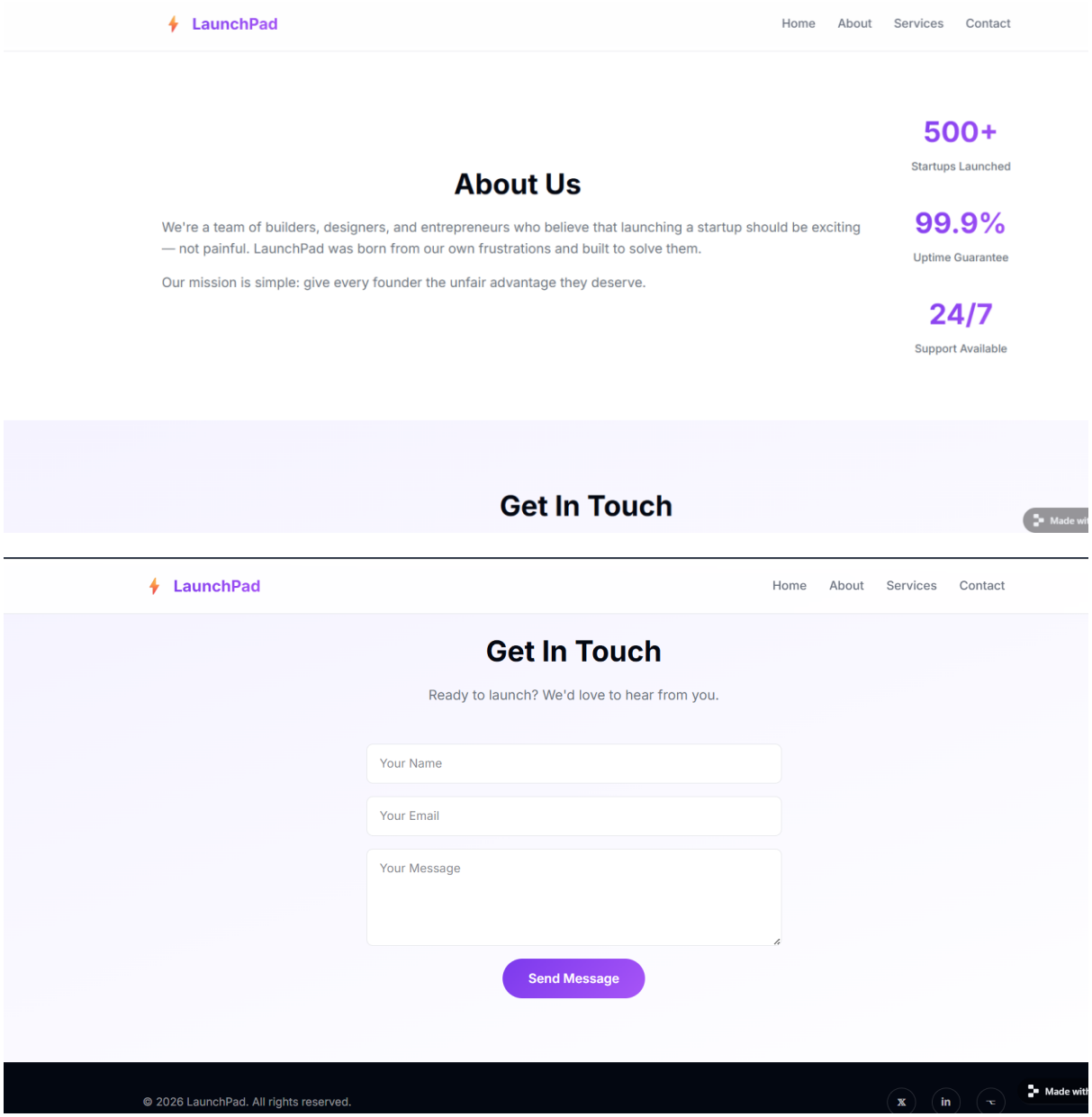
The page must display correctly in Replit Webview immediately after clicking Run.

Output:


LaunchPad
Home
About
Services
Contact
Build Something
Amazing Today
LaunchPad gives your startup the tools, speed, and confidence to go from idea to product faster than ever before.
Get Started
Why Choose LaunchPad?
Everything you need to take your startup from zero to launch.
Made with Replit


LaunchPad
Home
About
Services
Contact
Why Choose LaunchPad?
Everything you need to take your startup from zero to launch.
Fast Deployment
Ship your product in record time with our streamlined deployment pipeline and zero-config setup.
Secure by Default
Enterprise-grade security built right in — no extra configuration needed to keep your data safe.
Scale Effortlessly
From your first user to your millionth, LaunchPad scales with you without missing a beat.


500+
Startups Launched
Made with Replit



Explanation:

This website is a responsive startup homepage designed using semantic HTML and CSS to provide a clean and structured layout.

It contains a header with navigation links, a main section with a hero area and feature cards, and a footer with basic company information.

The layout uses modern CSS techniques like Flexbox/Grid to organize content clearly. Media queries are applied so the design automatically adjusts for mobile, tablet, and desktop screen sizes.

The purpose of the website is to present a professional landing page that is visually consistent and usable across different devices.

Link: https://startup-homepage-builder--poojithavunnam.replit.app

Task-02:

Prompt:

Create a simple interactive web page using HTML, CSS, and JavaScript.

1. HTML:

Add a clear call-to-action button labeled "Get Started" or "Click Me".

Include a heading and short description above the button.

Use semantic and clean HTML structure.

2. JavaScript Functionality:

When the button is clicked:

• Display a visible response (either an alert message or a message shown on the page).

The interaction must happen without reloading the page.

Use an event listener (addEventListener) instead of inline onclick.

3. JavaScript Code Quality:

Add meaningful comments explaining:

How the button is selected

How the click event works

What happens when the button is clicked

Keep the code beginner-friendly and readable.

4. Styling:

Add basic CSS to make the button visually appealing.

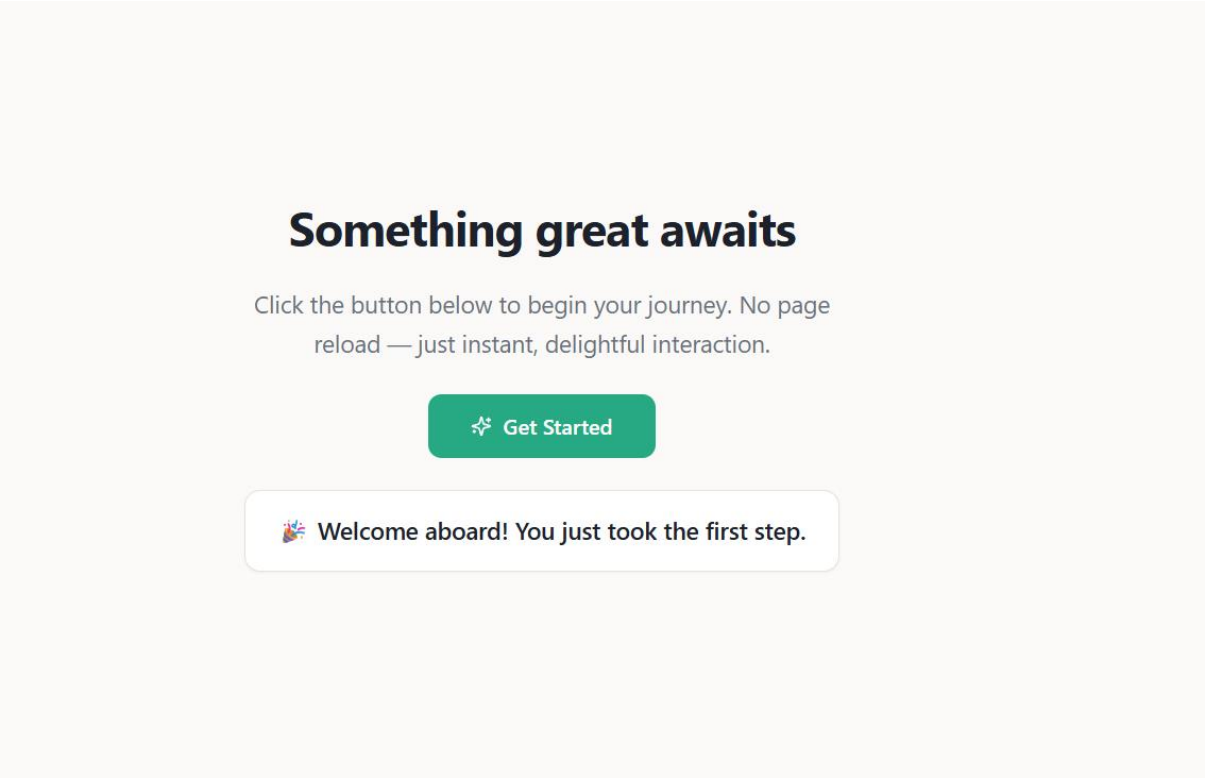
Include hover effects for better user interaction.

Provide one complete HTML file that runs directly in Replit.

Place CSS inside a <style> section and JavaScript inside a <script> section.

Ensure the page works immediately after clicking Run.

Output:



Explanation:

This web page includes an interactive call-to-action button created using HTML for structure and CSS for styling.

JavaScript is used to add functionality so the button responds when a user clicks it.

An event listener detects the click action and triggers a visible response such as an alert or on-screen message.

The interaction happens dynamically without reloading the page, improving user experience.

The JavaScript code contains meaningful comments that explain how the button is selected and how the click event works.

This demonstrates basic web interactivity by combining HTML, CSS, and JavaScript together.

Link: https://gentle-click-app.lovable.app

Task-03:

Prompt:

Create a responsive contact form using HTML, CSS, and JavaScript with client-side validation.\

FORM REQUIREMENTS:

- Include the following input fields:

  1. Name (text input)
  2. Email (email input)
  3. Message (textarea)

- Add a Submit button.
- Use semantic HTML structure.

VALIDATION LOGIC (JavaScript):

Prevent form submission if any field is empty.

Validate email format using a regular expression.

Show clear inline error messages below each field when input is invalid.Remove error messages automatically when the user corrects the input.

If all inputs are valid, display a success message on the page.

Do NOT reload the page (use event.preventDefault()).

CODE QUALITY:

Use addEventListener for form submission.

Write clean, beginner-friendly JavaScript.

Add meaningful comments explaining:

- form selection
- validation checks
- error message handling
- success message logi

STYLING:

Add simple CSS for a clean layout.

Highlight invalid fields with a red border.

Make the form centered and readable.

OUTPUT

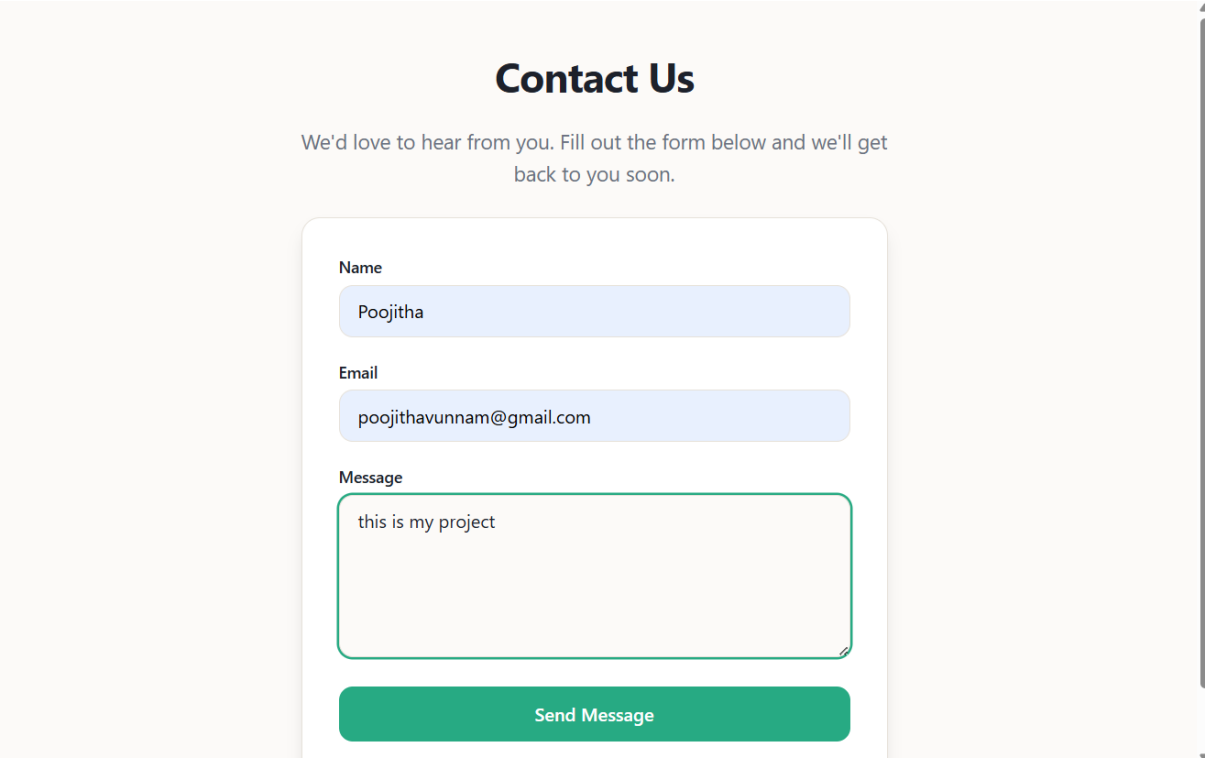
Provide ONE complete HTML file named "index.html".

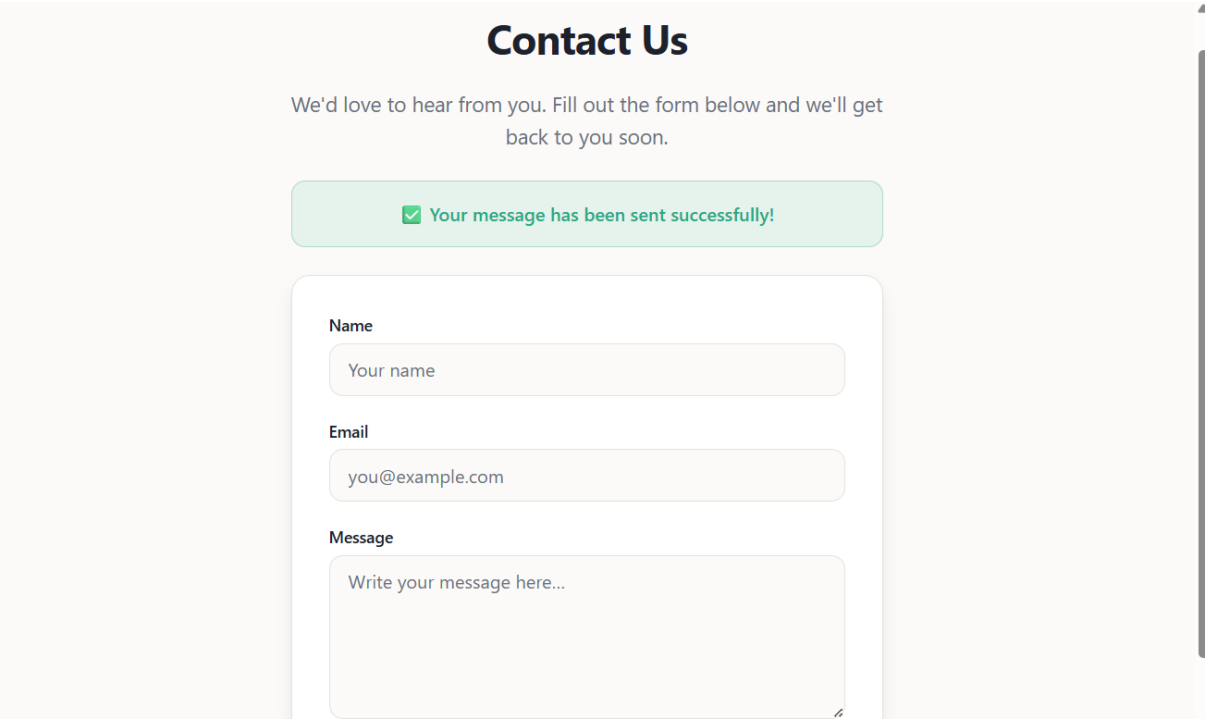
Include CSS inside a <style> section.

Include JavaScript inside a <script> section.

The page must run immediately in Replit preview after clicking Run.

Output:





Explanation:

This contact form is designed to collect user queries through fields such as name, email, and message using HTML for structure and CSS for layout and

styling. JavaScript is used to perform client-side validation to ensure that users enter valid information before submitting the form. The validation checks whether any field is left empty and verifies that the email follows a proper format. If invalid data is entered, clear error messages are displayed directly below the corresponding input fields to guide the user. When all inputs are correct, a success message appears without reloading the page, providing a smooth and interactive user experience.

Link: **https://contact-form-client-slide.lovable.app**

Task-04:

Prompt:

Create a simple dynamic product list web page using HTML, CSS, and JavaScript.

HTML REQUIREMENTS:

- Add a heading titled "Product List".

- Create an unordered list (<ul>) that displays product items.

- Include two buttons:

  • "Add Item"

  • "Remove Item"

- Add an input field where the user can type a product name to add.

JAVASCRIPT FUNCTIONALITY:

- When the "Add Item" button is clicked:

  • Read the value from the input field.

  • Add a new list item dynamically to the list using DOM manipulation.

  • Clear the input field after adding.

- When the "Remove Item" button is clicked:

  • Remove the last item from the list.

- All updates must happen instantly without reloading the page.

CODE QUALITY:

- Use addEventListener for button clicks.
- Write clean and beginner-friendly JavaScript.
- Add meaningful comments explaining:
  • element selection
  • event handling
  • how items are added to the DOM
  • how items are removed

STYLING:

- Add basic CSS to make the layout centered and readable.
- Style buttons with hover effects.

OUTPUT RULES:

- Provide ONE complete HTML file named "index.html".
- Include CSS inside a <style> section.
- Include JavaScript inside a <script> section.
- The page must run immediately in Replit preview after clicking Run.

Output:

Product List

Add or remove products from the list below.

CPU

Add Item

- Laptop
- Headphones
- Keyboard

Remove Last Item

Product List

Add or remove products from the list below.

Enter product name...

Add Item

- Laptop
- Headphones
- Keyboard
- CPU

Remove Last Item

Add or remove products from the list below.

CPU

Add Item

- Laptop
- Headphones
- Keyboard

Remove Last Item

Explanation:

This web page demonstrates dynamic list management using HTML, CSS, and JavaScript to handle product items interactively. An unordered list is used to display products, while buttons allow users to add new items or remove existing ones. JavaScript manipulates the DOM to update the list instantly when a button is clicked, without refreshing the page. The add function inserts a new list element based on user input, and the remove function deletes the last item from the list. This approach improves user experience by making the page responsive and interactive while keeping the code clear and maintainable.

Link: https://dynamic-list-management.lovable.app

Task-05

Prompt:

Create a modal popup notification web page using HTML, CSS, and JavaScript.

HTML REQUIREMENTS:

Add a button labeled "Open Modal".

Create a modal popup structure that includes:

- A modal container
- A message/title inside the modal
- A close button (× icon or "Close" button)

- Include a background overlay behind the modal.

CSS REQUIREMENTS:

- Style the modal so it appears centered on the screen.

- Add a semi-transparent dark overlay covering the entire page.

- Hide the modal by default.

- Add smooth transition or fade-in effect when opening and closing.

- Ensure the popup clearly overlays existing page content.

JAVASCRIPT FUNCTIONALITY:

- Clicking "Open Modal" should display the modal and overlay.
- Modal should close when:
  - The close button is clicked
  - The user clicks outside the modal (on the overlay)
- Use addEventListener for all interactions.
- Do not reload the page.
- Add meaningful comments explaining:
  - element selection
  - open modal logic
  - close modal logic
  - overlay click handling

OUTPUT

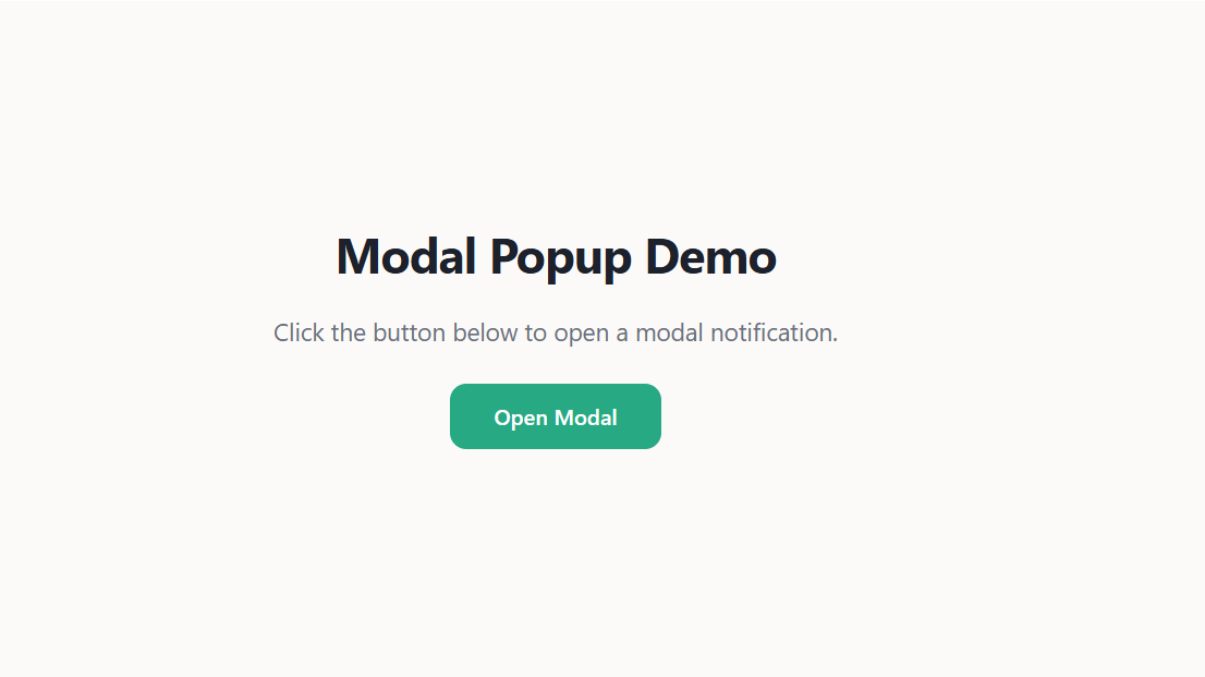
Provide ONE complete HTML file named "index.html".

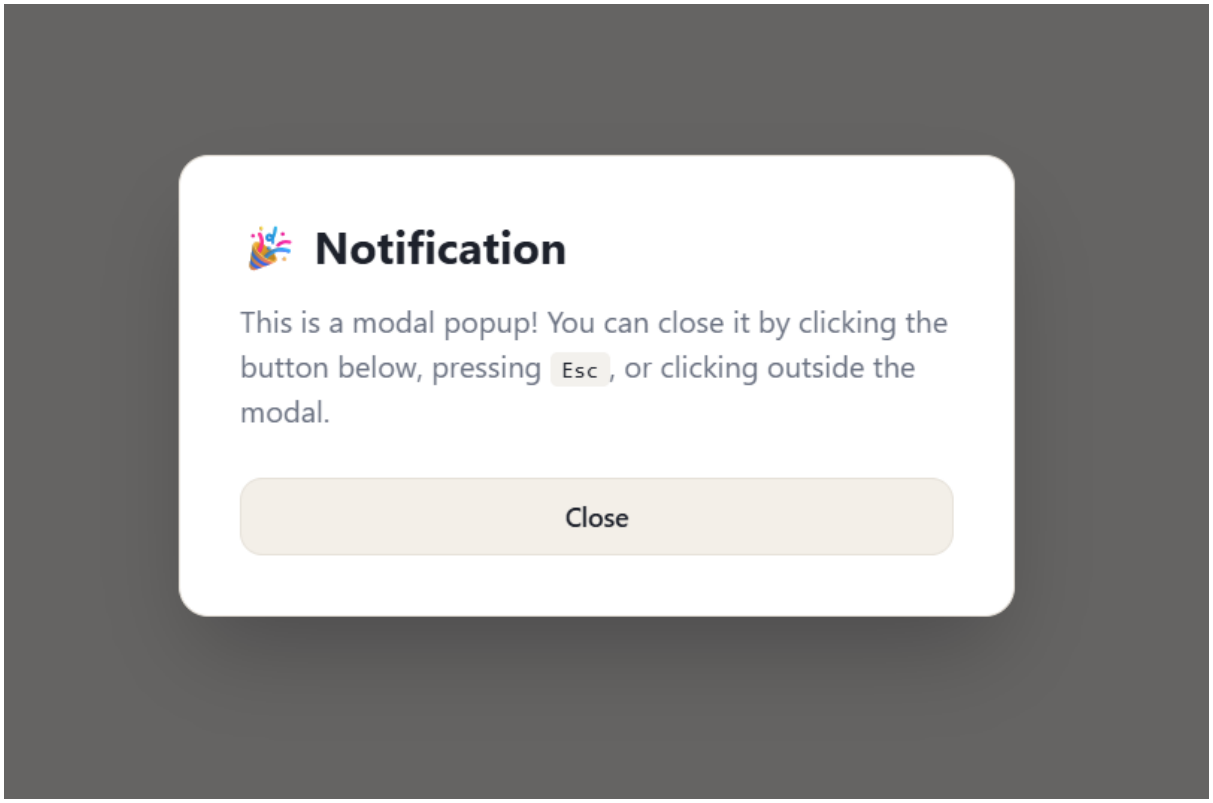
Include CSS inside a <style> section.

Include JavaScript inside a <script> section.

The page must run immediately in Replit preview after clicking Run.

Output:





Explanation:

This web page implements a modal popup used to display notifications or announcements to users. The modal is created using HTML for structure and CSS for styling, including a semi-transparent background overlay that highlights the popup while dimming the rest of the page. JavaScript controls the

behavior of the modal by opening it when the user clicks the “Open Modal” button. The popup can be closed either by clicking the close button or by clicking outside the modal on the overlay. All interactions happen dynamically without reloading the page, providing a smooth and user-friendly experience.

Link: **https://modal-popup.lovable.app**